

LABORATORY OBSERVATION / RECORD

Name: Kottana Roshini

Roll No.: A24126552263

Branch and Section: CSM-B

Course Name: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 03

1. TITLE

Write a Program Using JavaScript (Arrays, Functions)

2. AIM

To study JavaScript arrays and functions by performing array operations, using built-in array methods, creating different types of functions, and handling asynchronous data.

3. OBJECTIVE

The objectives of this experiment are:

- To understand how arrays are created and manipulated in JavaScript.
- To use built-in array methods such as push, pop, shift, unshift, map, filter, reduce, find, findIndex, includes, indexOf, slice, splice, sort and flat.
- To understand array/object destructuring and the spread operator.
- To create functions using different styles: function declarations, function expressions, arrow functions and implicit-return arrow functions.
- To understand asynchronous functions using async/await and fetch().
- To execute and verify the output of JavaScript programs using Node.js.

4. SYSTEM REQUIREMENTS

- Hardware: Processor (e.g., Core i5), RAM (e.g., 8GB), Storage.
- Software: Operating System (e.g., Windows 11 / Linux), Code Editor (e.g., VS Code), Browser (e.g., Chrome), Runtime (Node.js).

5. THEORY

An array in JavaScript is an ordered collection of values that can be created using array literals ([]) or the Array constructor. JavaScript provides many built-in methods to add, remove, search, transform and sort array elements.

A function is a reusable block of code. JavaScript supports several ways of defining functions: function declarations, function expressions, and arrow functions (including implicit-return one-liners). Arrow functions preceded by the async keyword allow the use of await to pause execution until a Promise (such as one returned by fetch()) resolves, without blocking the rest of the program.

Destructuring allows values to be unpacked from arrays or objects into distinct variables, and the spread operator (...) allows the elements of one array to be expanded into another.

6. ALGORITHM / PROCEDURE

- Create a new Node.js project and open it in VS Code.
- Create a file named Arrays.js to demonstrate array creation and built-in array methods.
- Perform basic operations: push, pop, unshift and shift on the array.
- Apply higher-order methods: map, filter and reduce.
- Use search methods: find, findIndex, includes and indexOf.
- Perform slicing and splicing operations, and sort an unsorted array.
- Demonstrate object destructuring and the spread operator on arrays.
- Use the flat() method to flatten a nested array.
- Create a file named Functions.js to demonstrate different function types.
- Define a normal function, a function expression, an arrow function and an implicit-return arrow function.
- Demonstrate a built-in function (Math.max).
- Define an async function that uses await with fetch() to retrieve data from an API.
- Define a multi-parameterized function and call it with arguments in order.
- Run both files using Node.js and verify the output.

7. PROGRAM

Arrays.js

```
//Note - Creation of an array
const arr = [3,7,11,15,19,23]
    //Constructor creation
const arr2 = new Array(3,7,11,15,19,23)
console.log(arr)
console.log(arr2)
arr.push(99)
console.log("pushed 99:"+["+arr+"])
arr.pop()
console.log("popped:"+["+arr+"])
arr.unshift(1)
console.log("unshifted 1:"+["+arr+"])
arr.shift()
console.log("shifted 1:"+["+arr+"])

//The Holy Trinity (Mid Tier)
//map function in array
const newarr = arr.map(
    (num)=>{return num * 5}
)
console.log("mapped arr"+newarr)
const booleanArr = arr.filter(
    (num)=>{return num%2 === 0}
)
console.log("booleanArr:"+booleanArr)
const total = arr.reduce((val1_starts_With_zero,val2_starts_with_first_element)=>{return
```

```

val1_starts_With_zero + val2_starts_with_first_element},0)
console.log("total (reduce sum):"+total)

//Finding Stuff
function greater(num){
    return num>10;
}
console.log("Find Elements greater than 10",arr.find(greater))
console.log("Find index for number greater than 10",arr.findIndex(greater))
console.log("Does 9 is there in arr",arr.includes(9))
console.log("index of 15 "+arr.indexOf(15))

//Slicing & Dicing
let sliced_arr = arr.slice(1,3)
console.log("sliced array",sliced_arr)
let spliced_arr = arr.splice(2,2)
console.log("spliced arr ",spliced_arr)
let unsorted_arr = [8,2,9,4]
console.log("Unsorted array:",unsorted_arr)
console.log("sorted array",unsorted_arr.sort())

//God Mode
// Destructuring
const person = {
    firstName: "Roshini",
    lastName: "Kottana",
    age: 20
};
console.log("Before Destructuring:",person)
let {firstName, lastName} = person;
console.log("After Destructuring - firstName:",firstName)
console.log("After Destructuring - lastName:",lastName)
console.log("Both Names:",{firstName,lastName})

//Spread Operator
const first = [5,10,15]
const second = [20,25,30]
console.log("first and second",first,second)
const combined = [...first,...second]
console.log("Combined:",combined)

//flat
const Fa = [2,[6,8]]
console.log("Before flat:",Fa)
console.log("After Flat:",Fa.flat())

```

Functions.js

```

// Normal way of function
// You can call it up here!
console.log(greetOG("Roshini"));

function greetOG(name) {
    return `Sup, ${name}`;
}

// Function Expression
const greetMillennial = function(name) {
    return `Hello, ${name}`;
};
console.log(greetMillennial("Roshini"));

```

```
// Arrow Function
const greetGenZ = (name) => {
  return `Yo, ${name}`;
};
console.log(greetGenZ("Roshini"));

// Implicit Return (one-liner arrow function)
const greetFast = name => `Yo, ${name} no cap`;
console.log(greetFast("Roshini"));

// Built-in Function
// Math.max() is a built-in function.
// You just hand it numbers and it does the math behind the scenes.
const highestScore = Math.max(72, 88, 95);
console.log(highestScore); // Output: 95

// Async Function
const fetchServerData = async () => {
  console.log("1. Sending request to server...");

  try {
    // await pauses THIS function until the fetch finishes,
    // while the rest of the app keeps running.
    const response = await fetch("https://jsonplaceholder.typicode.com/users/2");
    const data = await response.json();
    console.log("2. Data secured!", data.name);
  } catch (err) {
    console.log("2. Request failed (no network access in this environment).");
  }
};

fetchServerData();
console.log("3. Doing other stuff while waiting...");

// Multi-parameterized function
const trackStormSystem = (windSpeed, pressure, isLandfall) => {
  if (windSpeed > 119 && isLandfall === true) {
    return `Red Alert: Severe cyclone making landfall. Pressure at ${pressure}hPa.`;
  }
  return "System is stable. Keep monitoring.";
};

// Arguments must be passed in the exact same order as the parameters
const status = trackStormSystem(150, 940, true);
console.log(status);
```

8. STEPS TO EXECUTE

- Open VS Code.
- Click on Terminal.
- Click on New Terminal.
- Type node <your .js file> (e.g., node Arrays.js).
- Press Enter to view the output.

9. OUTPUT

Output 1: Arrays.js

```

$ node Arrays.js
[ 3, 7, 11, 15, 19, 23 ]
[ 3, 7, 11, 15, 19, 23 ]
pushed 99:[3,7,11,15,19,23,99]
popped:[3,7,11,15,19,23]
unshifted 1:[1,3,7,11,15,19,23]
shifted 1:[3,7,11,15,19,23]
mapped arr15,35,55,75,95,115
booleanArr:
total (reduce sum):78
Find Elements greater than 10 11
Find index for number greater than 10 2
Does 9 is there in arr false
index of 15 3
sliced array [ 7, 11 ]
spliced arr [ 11, 15 ]
Unsorted array: [ 8, 2, 9, 4 ]
sorted array [ 2, 4, 8, 9 ]
Before Destructuring: { firstName: 'Roshini', lastName: 'Kottana', age: 20 }
After Destructuring - firstName: Roshini
After Destructuring - lastName: Kottana
Both Names: { firstName: 'Roshini', lastName: 'Kottana' }
first and second [ 5, 10, 15 ] [ 20, 25, 30 ]
Combined: [ 5, 10, 15, 20, 25, 30 ]
Before flat: [ 2, [ 6, 8 ] ]
After Flat: [ 2, 6, 8 ]

```

Output 2: Functions.js

```

$ node Functions.js
Sup, Roshini
Hello, Roshini
Yo, Roshini
Yo, Roshini no cap
95
1. Sending request to server...
3. Doing other stuff while waiting...
Red Alert: Severe cyclone making landfall. Pressure at 940hPa.
2. Request failed (no network access in this environment).

```

10. RESULT

Thus, JavaScript array operations and different types of functions were successfully implemented and executed using Node.js. Array methods such as push, pop, map, filter, reduce, find, slice, splice, sort and flat were verified, along with function declarations, expressions, arrow functions and asynchronous functions using async/await.